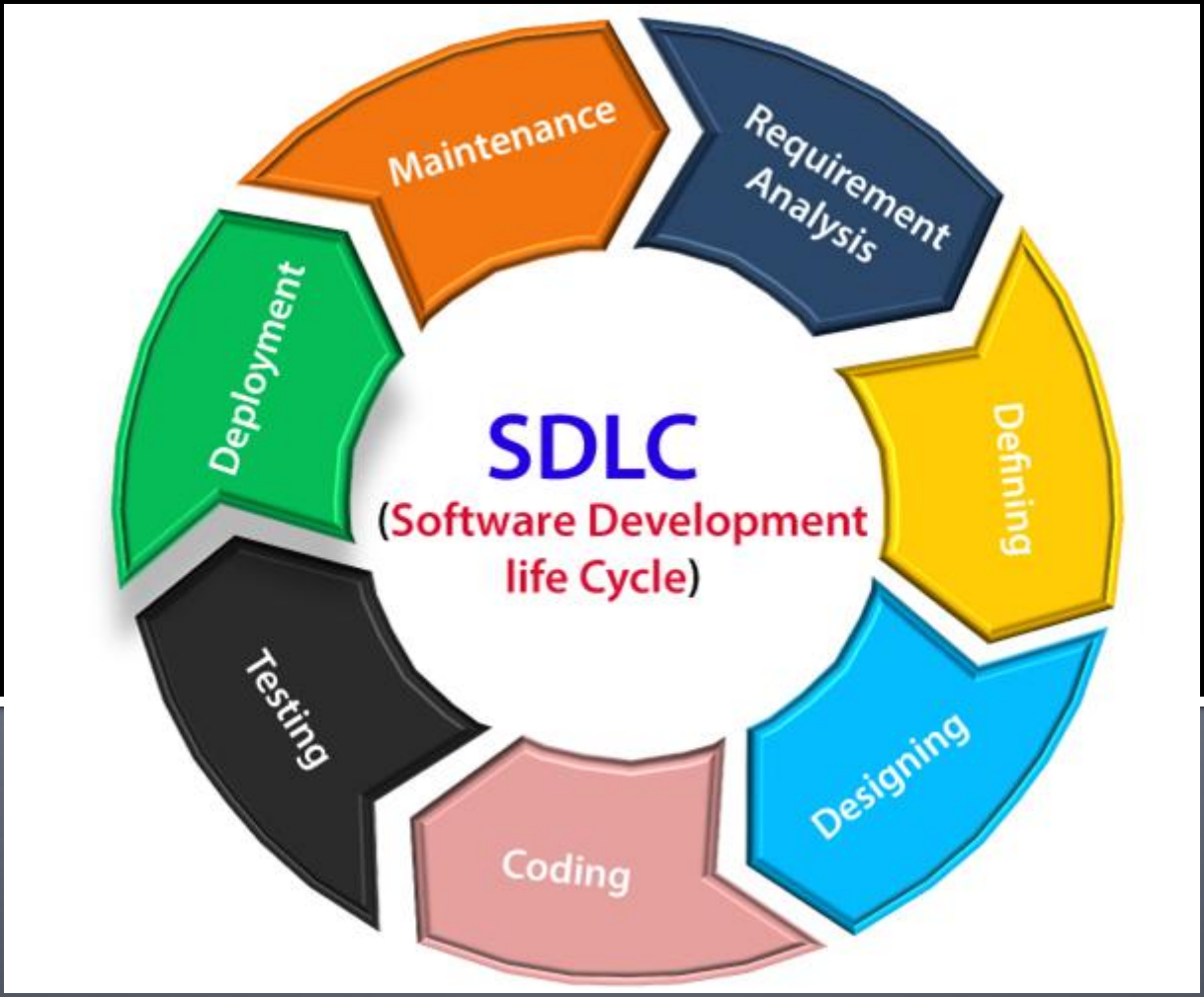
SOFTWARE DEVELOPMENT LIFECYCLE

Software Development Life Cycle (SDLC)



SDLC Model

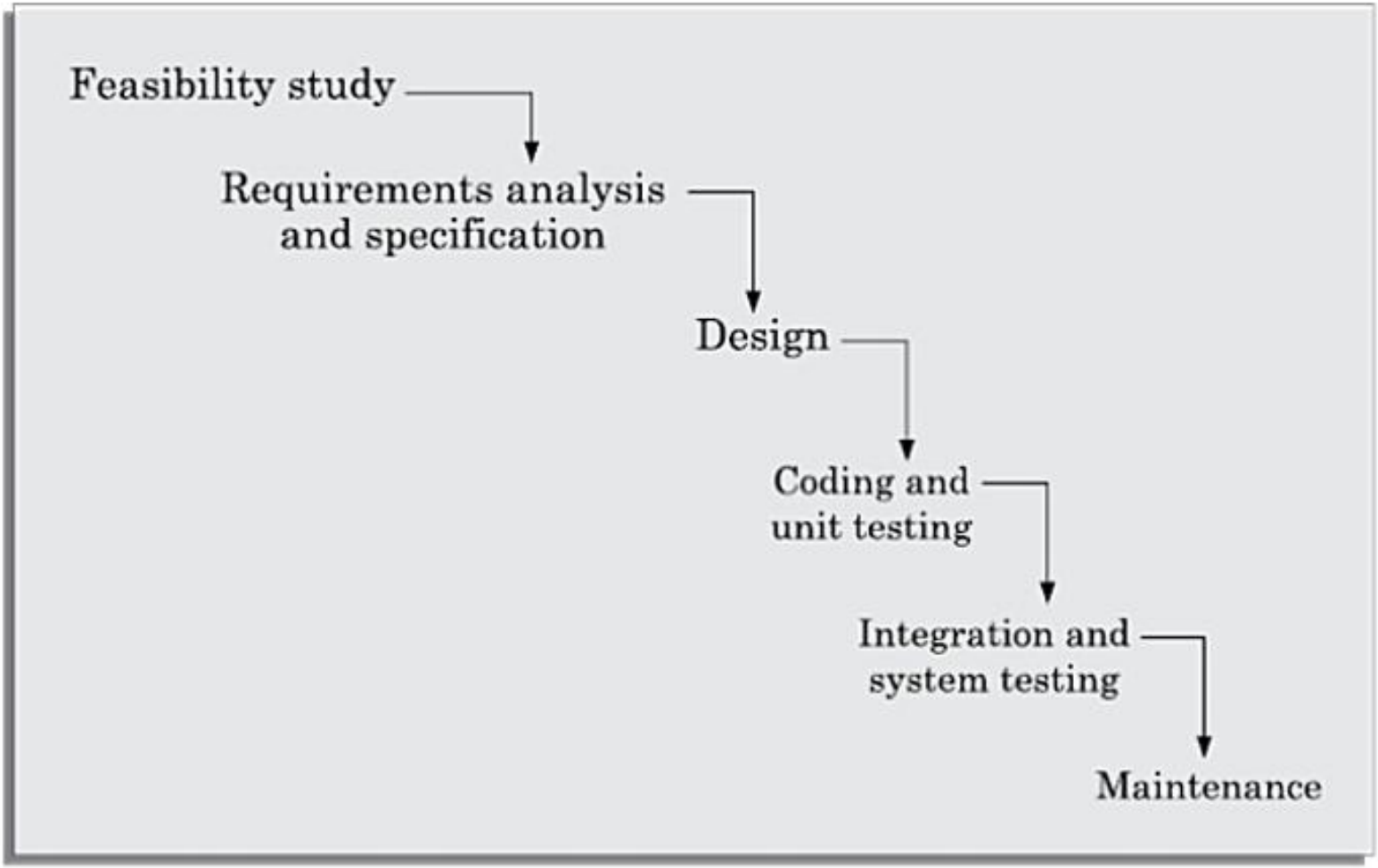
*"A **software development life cycle (SDLC)** model describes the different activities that need to be carried out for the software to evolve in its life cycle"*

- Remember that: There is **no one-size-fits-all approach** to software development.
- However, some steps are usually common: Requirement Analysis, Design, Implementation (Coding), Testing, **Deployment/ Maintenance**

Software Process Models

- **Prescriptive Process Models**
  - Prescribe a defined process and sequence of activities.
  - **Linear Models**
    - Water Fall Model
    - RAD Model
    - V- Model
  - **Incremental Models**
  - **Evolutionary Models**
    - **Prototyping Model**
    - **Spiral Model**
  - **Concurrent Model**
- **Agile**
  - Adapt the process as the project evolves

Classical Waterfall Model



Waterfall Strengths

- Easy to understand and implement.
- Provides a structured approach for inexperienced developers.
- Clearly defined phases and milestones.
- Requirements are fixed before development begins.
- Easy for managers to plan, monitor and control the project.
- Produces high-quality software when requirements are stable and quality is prioritized over cost or delivery time.

Waterfall Strengths

- Easy to understand, easy to use
- Provides structure to inexperienced staff
- Milestones are well understood
- Sets requirements stability
- Good for management control (plan, staff, track)
- Works well when quality is more important than cost or schedule

Waterfall Limitations

- **No feedback paths**
  - Once a phase is complete, the activities carried out in it and any artifacts produced in this phase are considered to be final and are closed for any rework.
  - This requires that all activities during a phase are flawlessly carried out.
- **Difficult to accommodate change requests:**
  - This model assumes that all customer requirements can be completely and correctly defined at the beginning of the project.
  - Difficult to accommodate any requirement **change requests** made by the customer after the **requirements specification** phase

Waterfall Limitations

- Inefficient error corrections:
  - This model defers integration of code and testing tasks until it is very late when the problems are harder to resolve.
- No overlapping of phases:
  - This model recommends that the phases be carried out sequentially—new phase can start only after the previous one completes.
  - Not realistic and leads to a large number of team members to stay idle for extended periods.

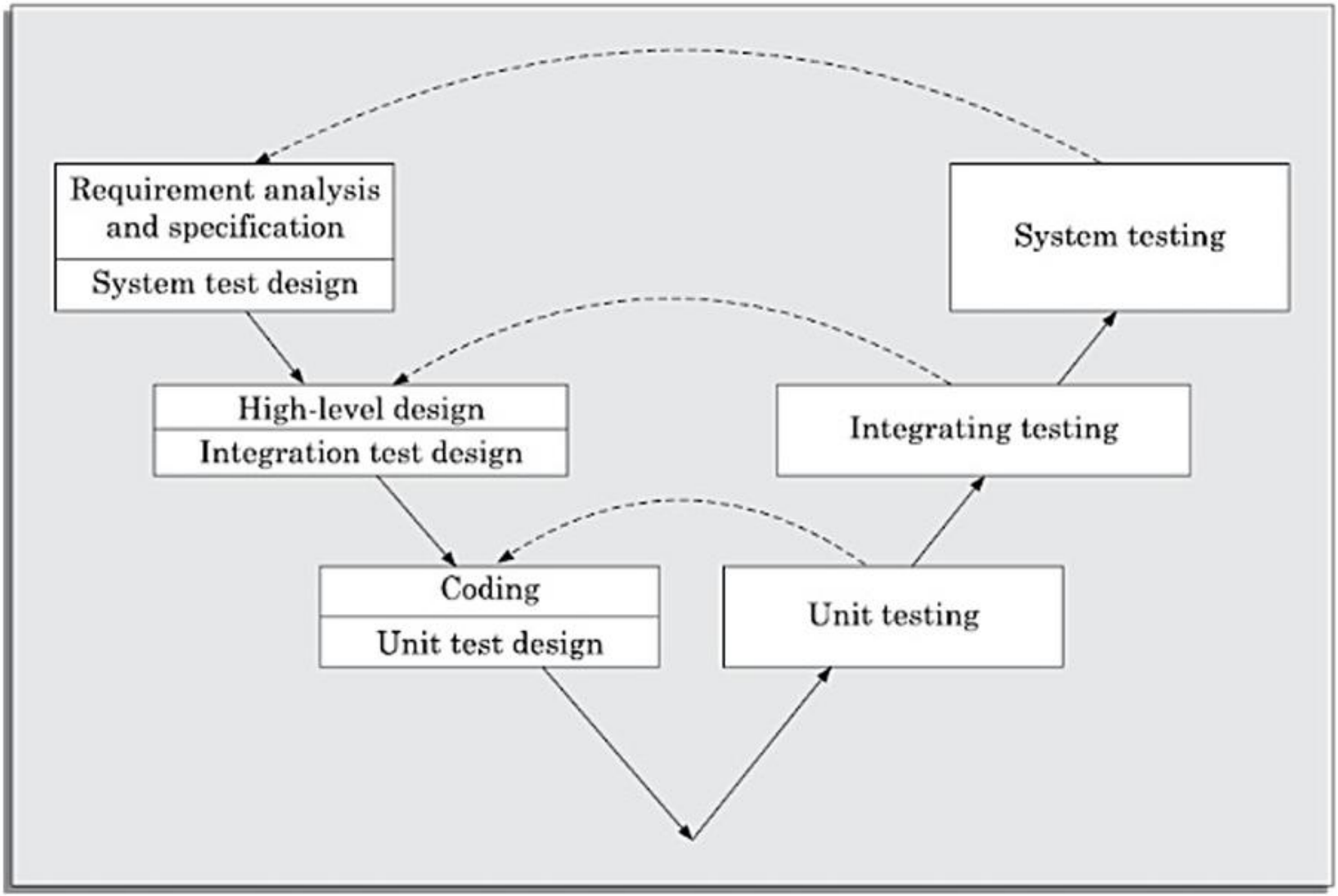
When to use the Waterfall Model

- Requirements are very well known
- Product definition is stable
- Technology is understood
- New version of an existing product
- Porting an existing product to a new platform.

V- Model

- **V-model** is a variant of the **waterfall model**.
- As is the case with the **waterfall model**, this model gets its name from its visual appearance
- In this model **verification and validation** activities are carried out throughout the development life cycle, and therefore the chances bugs in the work products considerably reduce.
- This model is therefore generally considered to be suitable for use in projects concerned with development of **safety-critical** software that are required to have **high reliability**.

V-Model


Requirement analysis and specification
System test design
System testing
High-level design
Integration test design
Integrating testing
Coding
Unit test design
Unit testing

V-Model

- There are two main phases—
  - development (Left phase) and
  - **validation phase** (Right phase).
- In each **development phase**, along with the development of a work product, **test case design** and the plan for testing the work product are carried out
- The actual testing is carried out in the **validation phase**.

V-Model Strengths

- Overall faster product development
  - Before testing phase starts significant part of the testing activities, including **test case design** and **test planning**, is already complete.
- Quality of the test cases are usually better
  - Test cases are designed when the schedule pressure has not built up
- More efficient manpower utilisation
  - Test team is reasonably kept occupied throughout the development cycle
- Test team can carry out effective testing of the software
  - Test team is associated with the project from the beginning. Therefore they build up a good understanding of the development artifacts

V-Model Weaknesses

- Being a derivative of the **classical waterfall model**, this model inherits most of the weaknesses of the **waterfall model**.
- Does not easily handle concurrent events
- Does not handle iterations or phases
- Does not easily handle dynamic changes in requirements
- Does not contain risk analysis activities

When to use the V-Shaped Model

- In this model **verification and validation** activities are carried out throughout the development life cycle, and therefore the chances of bugs in the work products considerably reduce.
- Excellent choice for **safety-critical** systems requiring **high reliability** – hospital patient control applications
  - **Safety-Critical Embedded Systems:** Used in industries where failures can result in injury or severe loss (e.g., aerospace control systems, automotive braking systems, pacemakers).
- All requirements are known up-front

Iterative Waterfall Model

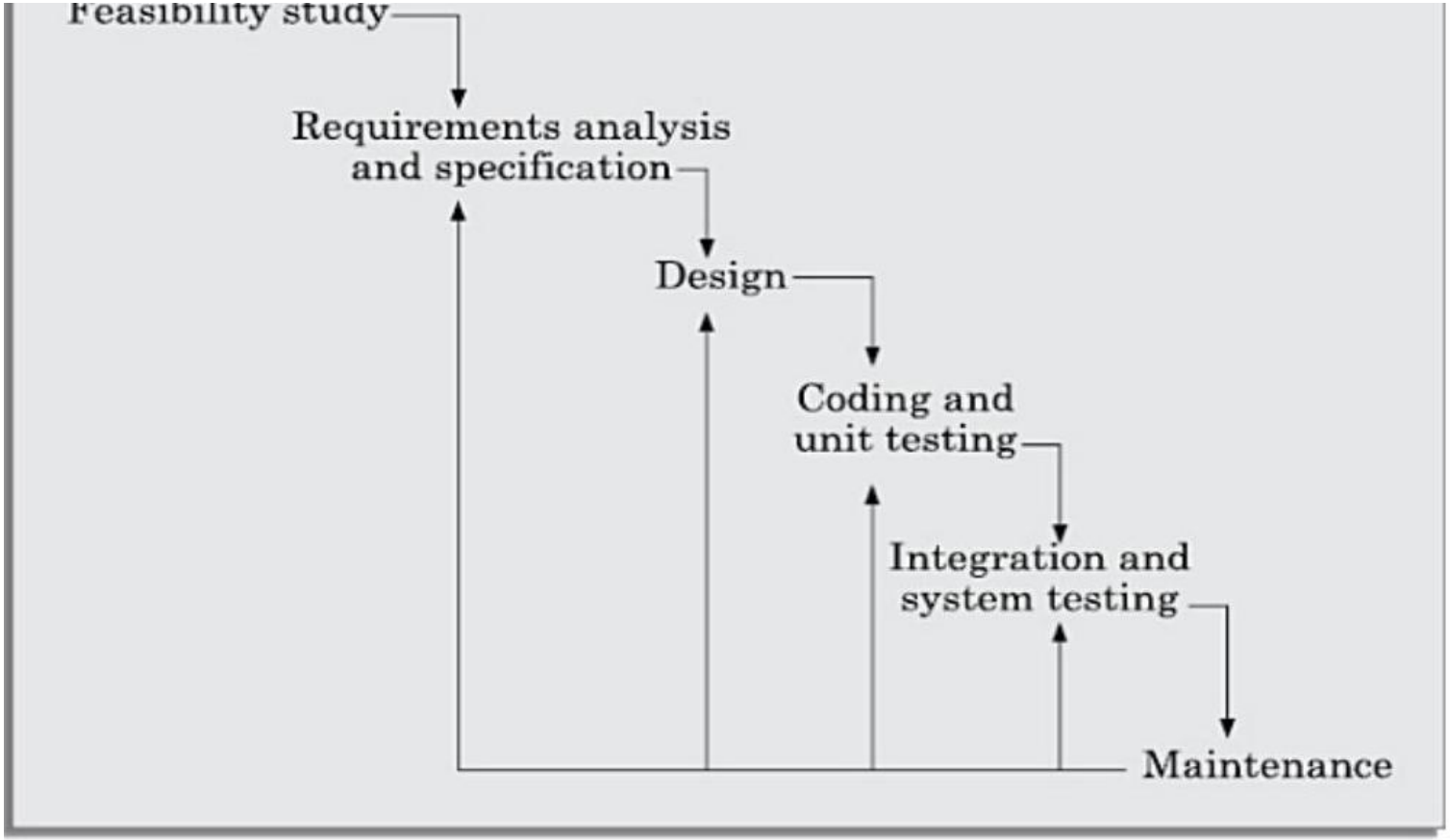


Figure 2.3: Iterative waterfall model.

Iterative Waterfall Model

- Key modification over Classical Waterfall: **feedback paths** from every phase back to its preceding phases
- Allows correcting errors detected in later phases by revisiting earlier phases
- **No feedback path** to feasibility stage - once a project is accepted, legal/moral obligations prevent abandoning it
- **Phase Containment of Errors**: detecting errors as close to their point of commitment as possible
  - Rigorous review of documents at the end of each phase helps achieve **phase containment**
- **Phase Overlap**: phases overlap in practice because early completers start next phase and rework causes re-entry into prior phases

Iterative Waterfall Model: Shortcomings

- Difficult to accommodate **change requests**: requirements must be frozen before development starts
  - Once requirements are frozen, no scope for modifications - but later changes are almost inevitable
- **Incremental delivery** not supported: full software is delivered only after complete development
  - By the time software is delivered, the customer's business process may have changed substantially
- Error correction unduly expensive: **validation** delayed until complete development; defects incur heavy rework
- Limited **customer interactions**: only at project start and end - delivered software may not meet real needs
- Heavy weight: overemphasis on documentation reduces team efficiency

Prototyping Model

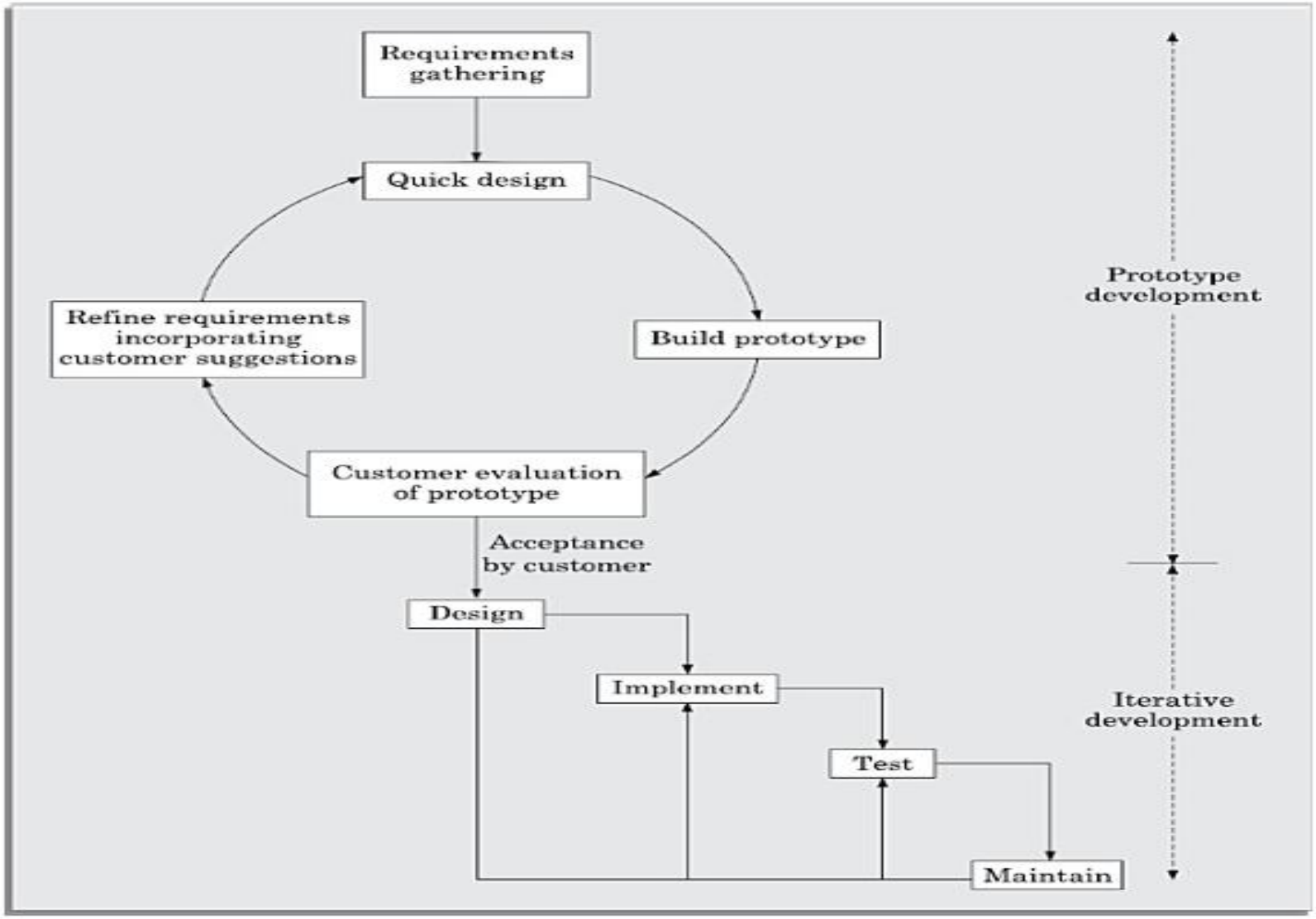


Figure 2.6: Prototyping model of software development.

Prototyping Model

- A **prototype** is a crude, limited implementation built quickly using shortcuts (dummy functions, table look-ups, 4GL tools)
- Useful for three types of projects:
  - **GUI development**: customers can evaluate screen layouts, dialogs, and reports visually rather than imagining them
  - Unclear technical solutions: prototyping helps examine technical risks (e.g., building a small compiler first)
  - Highly optimised software: plan to throw away the first version to build a better one (Brooks, 1975)
- **Prototype** code is usually thrown away; experience gained helps in the actual development
- **GUI** parts of a software system are almost always developed using the **prototyping model**

Prototyping Model: Life Cycle and Evaluation

- Life Cycle Activities:
  - **Prototype** Development: requirements gathering, quick design, build **prototype**, **customer feedback**, refine - repeat until approved
  - Iterative Development: after **prototype** approval, actual software built using the iterative waterfall approach
- Strengths:
  - Most appropriate for projects with technical and requirements-related risks
  - Reduces later **change requests** and associated redesign costs
- Weaknesses:
  - Increases cost for routine projects that do not suffer from significant risks
  - Effective only when risks can be identified upfront before development starts
  - Ineffective for risks identified later during the development cycle

Incremental Development Model

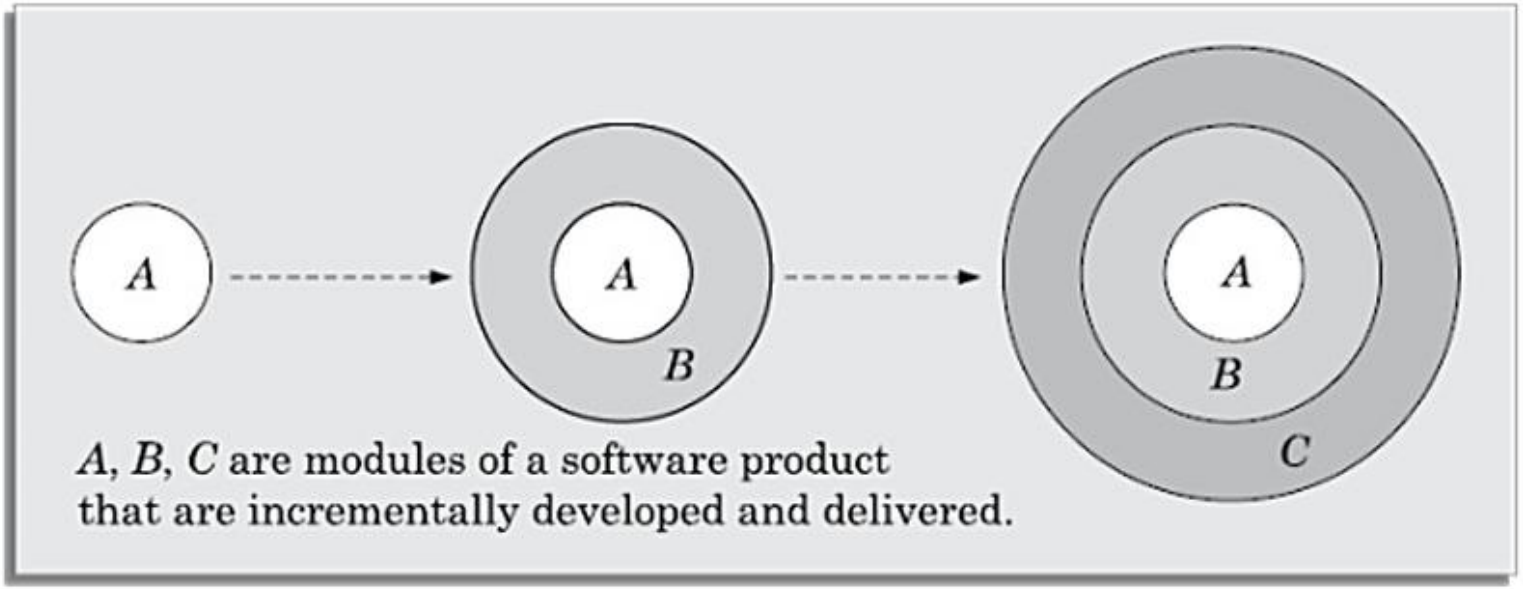
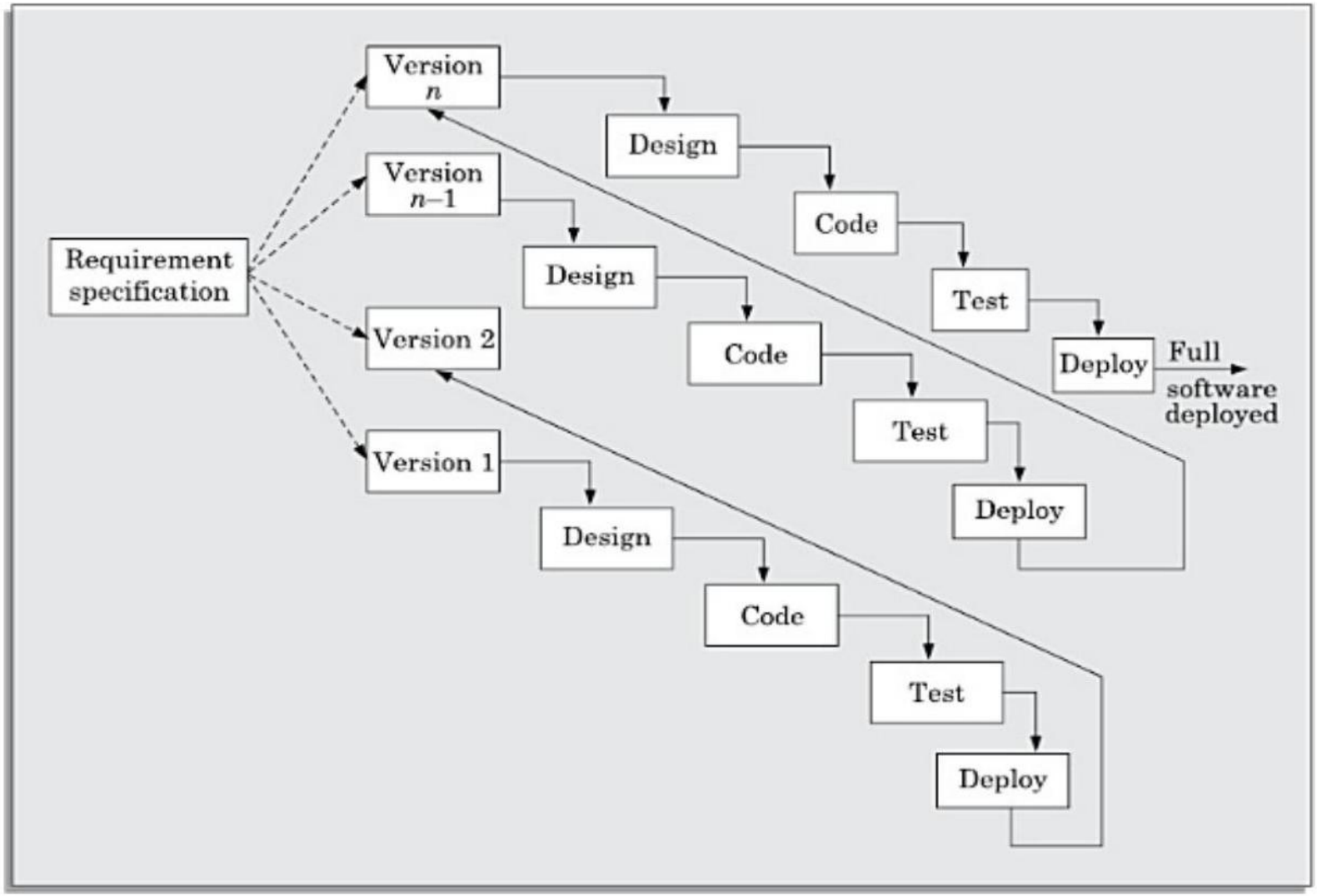


Figure 2.7: Incremental software development.

Incremental Development Model



Incremental Development Model

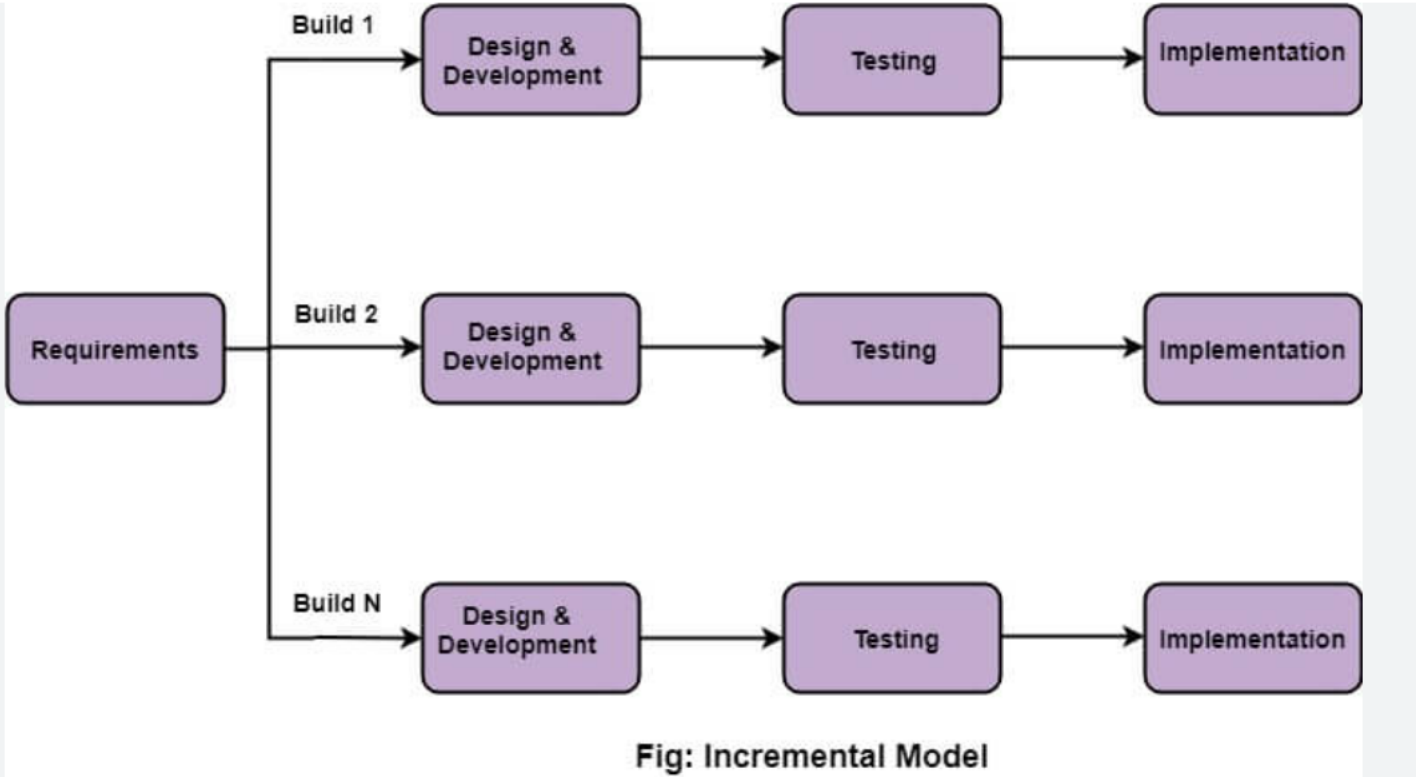


Fig: Incremental Model

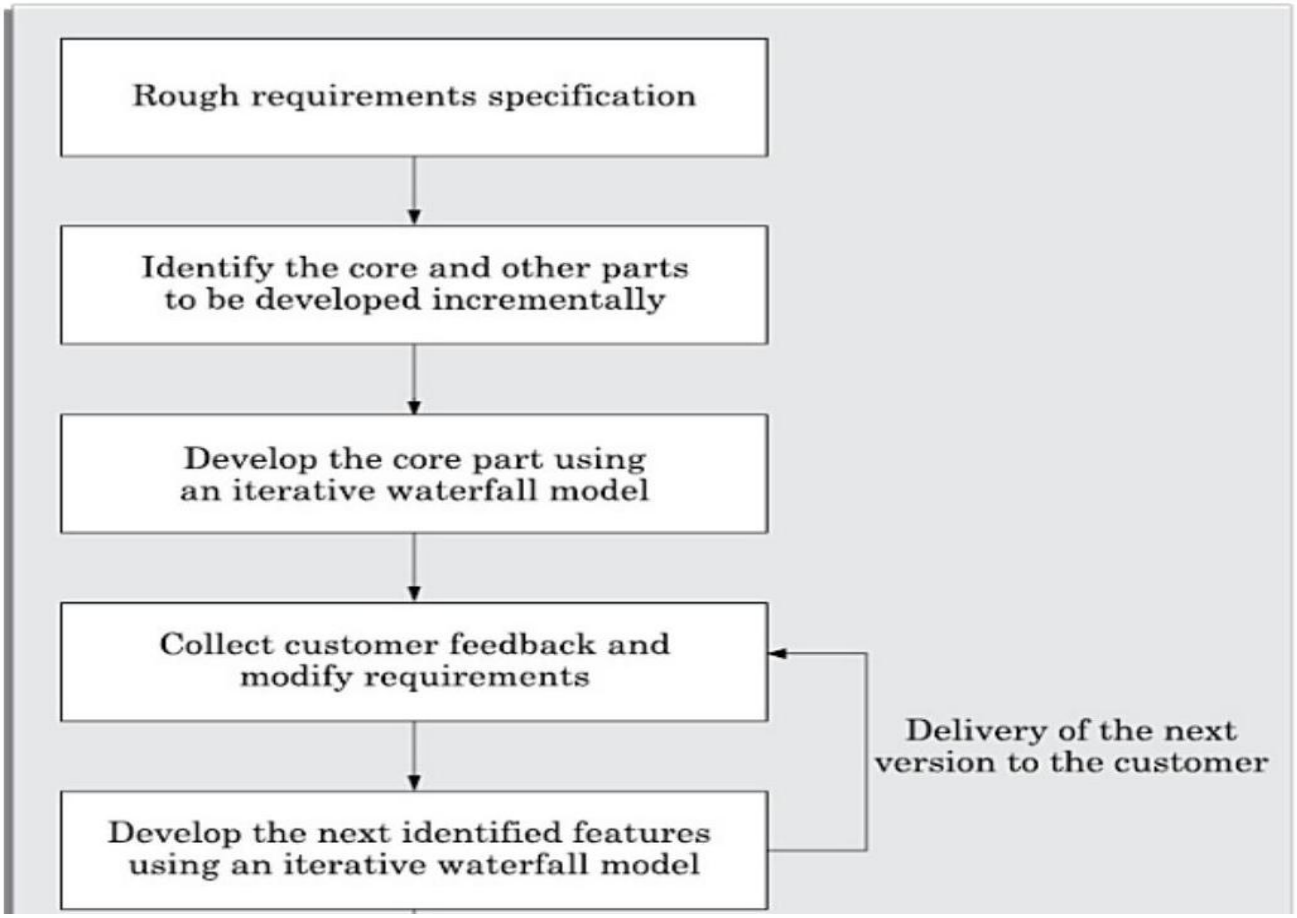
Incremental Development Model

- Also called **Successive Versions Model**
- First a simple working system with basic features is built and delivered to the customer
- Over successive iterations, more features are added and delivered until the full system is realised
- Requirements are split into modules/features for incremental construction and delivery
- Plan is made only for the next increment; no long-term plans, making change accommodation easier
- **Core features** (those not dependent on other features) are developed first
- Each incremental version is developed using the **iterative waterfall model**
- **Customer feedback** on each delivered version is incorporated into the next version

Incremental Model: Advantages

- **Error Reduction**:
  - **Core modules** are used by the customer from the beginning and therefore get thoroughly tested
  - Reduces chances of errors in the **core modules**, leading to greater reliability of the final product
- **Incremental Resource Deployment**:
  - Customer need not commit large resources all at once for the full system
  - Developing organisation is also saved from deploying large resources in one go
- Early partial delivery allows customers to start using the system and provide useful feedback sooner
- Unlike **Evolutionary Model**: in **Incremental Model**, complete requirements are specified and frozen upfront

Evolutionary Model



Evolutionary Model

- Also called **design a little, build a little, test a little, deploy a little model**
- Requirements, plan, estimates, and solution evolve over iterations - not frozen upfront
- Suitable for projects with unpredictable feature discovery and frequently changing requirements
- Advantages:
  - Effective elicitation of actual customer requirements through feedback on partial deliveries
  - Easy handling of **change requests**; reworks are smaller compared to sequential models
- Disadvantages:
  - Feature division can be non-trivial for small or tightly-coupled systems
  - Ad hoc design: without long-term planning, design may lack maintainability and optimality
- Best suited for large products, **object-oriented** development, and customer-preferred **incremental delivery**

Rapid Application Development (RAD) Model

- Proposed in the early 1990s to overcome the rigidity of the **waterfall model**
- Combines features of both prototyping and **evolutionary models**
- Unlike **prototyping model**, prototypes in RAD are NOT thrown away but enhanced into the final product
- Major goals:
  - Decrease time and cost of software development
  - Limit costs of accommodating **change requests**
  - Reduce the communication gap between customer and developers
- Development team of 5-6 members always includes a **customer representative**

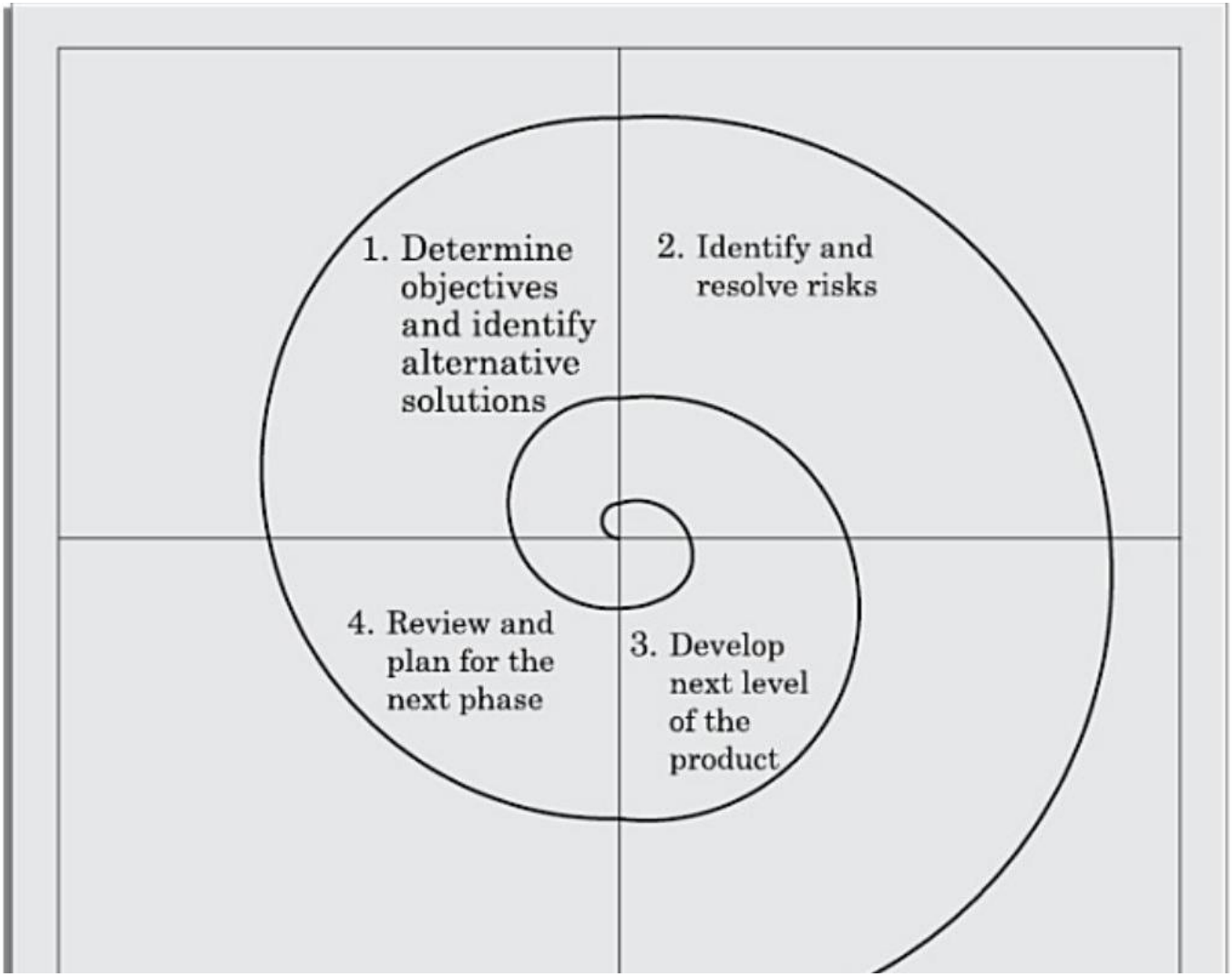
Rapid Application Development (RAD) Model

- The **Rapid Application Development (RAD) Model** is a software development model that emphasizes **rapid development and quick delivery** by using:
- Quick Prototyping
- Reusable components
- Incremental Releases
- User involvement
- Parallel development of modules
- Instead of spending months on planning and documentation, RAD focuses on **building working software quickly**.

RAD Model: Working and Applicability

- Working:
  - Development in short cycles called **time boxes**; each box implements a small increment of functionality
  - **Quick-and-dirty prototype** built each **time box**; customer evaluates and gives feedback; **prototype** is refined
  - Heavy **code reuse** and visual/specialised tools used for rapid **prototype** creation
- Suitable for:
  - Customised software with significant **code reuse** from existing similar applications
  - Non-critical software; projects with aggressive time schedules; large feature-rich software
- Not suitable for:
  - Generic products requiring optimal performance or reliability (e.g., operating systems, flight simulators)
  - Projects with no similar prior products - insufficient plug-in components for reuse

Spiral Model



Spiral Model

- Gets its name from its diagrammatic appearance - a spiral with multiple loops
- Each loop of the spiral is called a phase; the number of phases is decided dynamically by the project manager
- A prominent feature: handles unforeseen risks that show up during development, not just at the start
- Each phase is split into four quadrants:
  - Q1: Investigate and analyse objectives; identify risks; propose alternative solutions
  - Q2: Evaluate alternatives by building a **prototype** to resolve identified risks
  - Q3: Develop and verify the next level of the software product
  - Q4: Review the developed version with customer and plan the next spiral loop
- Radius of the spiral = cumulative project cost; angular progress = progress in current phase

Spiral Model: Strengths and Weaknesses

- Strengths:
  - Handles risks at every phase - not just at the beginning like the **prototyping model**
  - Acts as a **meta model** - subsumes all other models: waterfall (single loop), prototyping (risk reduction), evolutionary (incremental loops)
  - Best suited for large, technically challenging software prone to multiple unforeseen risks
- Weaknesses:
  - Complex to follow - requires knowledgeable, experienced staff
  - Not suitable for outsourced projects as risks must be continually assessed internally
  - Counterproductive for small or routine projects due to overhead (increases cost and time without providing significant benefits.)
- When to use: projects with many unknown risks that are likely to emerge during development

Agile Development Model

- Proposed in the mid-1990s to overcome the serious shortcomings of the **waterfall model**
- Core idea: fit the process to the project; remove unnecessary activities; avoid anything that wastes time and effort
- **Agile Manifesto** (2001) key principles:
  - **Working software** over comprehensive documentation
  - Frequent delivery of incremental versions in intervals of a few weeks
  - Customer **change requests** are actively encouraged and efficiently incorporated
  - **Face-to-face communication** preferred over formal documents
  - Continuous **customer interaction** more important than contract negotiation
- Team size deliberately kept small (5-9 members); always includes a **customer representative**
- **Time box**: fixed delivery date per iteration; team may reduce scope but not move the deadline

Agile Model: Evaluation and Applicability

- Strengths:
  - Highly flexible - **change requests** efficiently incorporated at any stage
  - Early and continuous delivery of **working software** as the measure of progress
  - Close customer collaboration ensures the developed software matches real requirements
  - **Pair programming** reduces bugs and improves code quality
- Weaknesses:
  - Lack of formal documentation can cause confusion, misinterpretation, and **maintenance** difficulties
  - Difficult to scale for large teams; teams at different locations need heavy daily communication
  - Not suitable for mission-critical or **safety-critical** systems

Agile Model: Evaluation and Applicability

- Popular agile models:
  - **Scrum :** In the scrum model, a project is divided into small parts of work that can be incrementally developed and delivered over time boxes that are called sprints.
  - **Extreme Programming** (XP) : XP is based on frequent releases (called iteration), during which the developers implement "user stories". User stories are similar to use cases but are more informal and are simpler.
  - Other frameworks are Crystal, Feature-Driven Development, Lean, Unified Process.
- When to use: small customised software projects with rapidly changing or unclear requirements

Choosing the Right SDLC Model

SDLC models provide structured frameworks for software development

- **Classical Waterfall Model**: simple and sequential; best for stable, well-known requirements
- **Iterative Waterfall Model**: adds **feedback paths**; most widely used in practice
- **V-Model**: pairs each **development phase** with **verification and validation**; ideal for **safety-critical** systems
- **Prototyping Model**: builds a quick **prototype** to resolve unclear requirements or technical risks
- **Incremental Model**: delivers **core features** first; grows the system over successive versions
- **Evolutionary Model**: both requirements and solution evolve over iterations; suits large, changing projects
- **RAD Model**: rapid delivery via **time boxes**, **code reuse**, and continuous **customer feedback**
- **Spiral Model**: handles unforeseen risks at every phase using prototyping in each loop
- **Agile Model**: customer collaboration, **working software** and **change requests** over rigid plans

No single model fits all projects — choice depends on requirements clarity, project size, and risk